

tbl_user_credit_transaction

Per-user credit movement journal · Phase 21.5

ตารางใหม่สำหรับเก็บ ประวัติทุกการเปลี่ยนแปลง credit ของ user — ทั้งตอนได้รับเงิน (topup จาก admin) และตอนใช้เงิน (usage จาก chat). ทุก event มี balance_before / balance_after เพื่อ audit ครบ; รองรับทั้ง Day view และ Month view จาก mockup dashboard ของลูกค้า.

Table name	tbl_user_credit_transaction	Type	1 table + 1 view
Risk	Low (additive only)	Rollback	DROP TABLE — data ไม่เสีย
Migration	phase21-005 + phase21-006	Code edits	2 จุดใน server.js

SECTION 01

ทำไมต้องสร้าง table นี้

จาก mockup dashboard ที่ลูกค้าออกแบบ ต้องการดู "history per user" ว่าใคร ได้รับ credit มาเท่าไหร่ และ ใช้ไป เท่าไหร่ ในช่วงเวลาที่เลือก (ดูทั้งแบบรายวัน + รายเดือน). ตอนนี้ระบบเรามีข้อมูลครบข้างขาดบ้าง:

	มีในระบบแล้ว	สิ่งที่ขาด
Topup	☐ admin → project pool	☐ admin → user (ขาด!)
Usage	☐ tbl_response (chat)	☐ มีครบ
Audit	☐ ปนใน tbl_action_admin	☐ query ยาก (JSONB)

ปัญหาหลัก: "admin จัดสรร pool → user" ไม่มีการบันทึกเป็น transaction — ตอนนี้แค่เห็นใน audit log ปนกับ action อื่นๆ. dashboard ที่ลูกค้าออกแบบจะ query ยากมาก.

SECTION 02

อธิบาย Schema ที่ละ column

ตารางนี้เก็บ 1 row ต่อ 1 transaction (ทุก credit movement = 1 row). ไม่มีการ aggregate ใน table — Day/Month view ทำที่ query level.

Column	Type	Required	หน้าที่	ตัวอย่าง
transaction_id	BIGSERIAL	auto	PK — รัน serial	1, 2, 3, ...
user_id	INT	NOT NULL	FK → tbl_user — user คนไหน	2 (user)
project_id	VARCHAR(100)	NULL	FK → tbl_project — context	proj_abap
transaction_type	VARCHAR(20)	NOT NULL	ประเภท transaction (4 ค่า — ใช้ 2 อันแรกก่อน, อีก 2 ส้ารอง)	topup / usage
amount	DECIMAL(12, 4)	NOT NULL	จำนวนเงิน มีเครื่องหมาย (+ = ได้รับ, - = ใช้)	+50.00 / -0.88
balance_before	DECIMAL(12, 4)	NULL	ยอดก่อน transaction (snapshot ตอน insert)	100.00
balance_after	DECIMAL(12, 4)	NULL	ยอดหลัง transaction (= before + amount)	99.12
ref_type	VARCHAR(20)	NULL	มาจากการทำอะไร — chat / admin_edit / admin_topup	chat
ref_id	BIGINT	NULL	id ของต้นทาง (session_id / action_id) — สำหรับ trace กลับ	42 (session)
note	TEXT	NULL	admin จดบันทึกได้ (เช่น "เดิมตามใบ PR2025-001")	"top-up Q4"
created_by	INT	NULL	admin user_id ที่ทำ (NULL = auto จาก chat)	1 (admin)
created_at	TIMESTAMP TZ	NOW()	เวลาที่เกิด transaction — ใช้ filter Day/Month	2025-10-28 14:22

Indexes ที่สร้างให้

เพื่อให้ query Day view + Month view + filter ตาม user/project ทำงานเร็ว:

ชื่อ index	Columns	ใช้สำหรับ
idx_uct_user_date	(user_id, created_at DESC)	"user คนนี้ดูประวัติย้อน"
idx_uct_project_date	(project_id, created_at DESC)	"project นี้สรุป transaction"
idx_uct_date	(created_at DESC)	"ทุก transaction วันนี้"
idx_uct_type_date	(transaction_type, created_at)	"แค่ topup / แค่ usage"

SECTION 03

Transaction Types — 4 ประเภท

Column transaction_type มี 4 ค่าที่กำหนดไว้ — เริ่มใช้ 2 อันแรก ก่อน (topup + usage) อีก 2 อันสำรองไว้รองรับ feature ในอนาคต (refund, admin adjustment) โดยไม่ต้องเปลี่ยน schema.

Type	Sign	ความหมาย	ตัวอย่าง	Active?
topup	+	admin เดิม credit ให้ user คนนี้	Admin ใส่ 100 ใน Edit User → +100	YES
usage	-	user ใช้ chat — หัก credit ตาม cost	User chat 1 turn cost ฿0.88 → -0.88	YES
adjustment	±	admin แก่ตรงๆ (เช่น โอน, แก้ข้อผิดพลาด)	Admin ปรับลด credit ที่เดิมผิด → -10	รอใช้
refund	+	คืน credit (เช่น chat fail, complaint)	User chat error ระบบ → คืน +0.88	รอใช้

SECTION 04

ตัวอย่างจริง — Flow + rows ใน table

สมมติ user คนหนึ่งใช้งานในรอบ 4 วัน — หมายความว่า rows ใน table จะหน้าตาเป็นแบบไหน:

#	When	What happened	DB action	Balance after
1	25/10 09:00	Admin Somchai เดิม 100 บาท ให้ user	INSERT topup +100	+100.00
2	26/10 14:30	User chat 1 turn (cost 0.88)	INSERT usage -0.88	99.12
3	27/10 10:15	User chat อีก 1 turn (cost 1.20)	INSERT usage -1.20	97.92
4	28/10 16:00	Admin เดิมเพิ่มอีก 50 บาท	INSERT topup +50	147.92

→ ใน tbl_user_credit_transaction จะมี 4 rows แบบนี้:

id	type	amount	before	after	ref	created_at
1	topup	+100.00	0.00	100.00	admin_edit#A1	2025-10-25 09:00
2	usage	-0.88	100.00	99.12	chat#42	2025-10-26 14:30
3	usage	-1.20	99.12	97.92	chat#42	2025-10-27 10:15
4	topup	+50.00	97.92	147.92	admin_edit#A2	2025-10-28 16:00

SECTION 05

ใช้ query ยิง — Day vs Month view

จุดเด่นของ design นี้คือ table เดียวรองรับทั้ง 2 views — ต่างกันแค่ การ aggregate:

Day view → ดูทุก transaction แยกแถว

```
SELECT created_at::date AS date,
       u.username,
       t.transaction_type AS type,
       ABS(t.amount)      AS amount
FROM tbl_user_credit_transaction t
JOIN tbl_user u USING (user_id)
WHERE created_at::date BETWEEN '2025-10-25' AND '2025-10-28'
ORDER BY created_at DESC;
```

→ ได้ผลลัพธ์ตรงกับรูป "Day view" ของลูกค้า

Month view → รวมตามเดือน + user + type

```
SELECT TO_CHAR(date_trunc('month', created_at), 'FMMonth YYYY') AS month,
       u.username,
       t.transaction_type AS type,
       SUM(ABS(t.amount))::numeric(12,2) AS total
FROM tbl_user_credit_transaction t
JOIN tbl_user u USING (user_id)
WHERE created_at >= '2025-10-01' AND created_at < '2025-12-01'
GROUP BY date_trunc('month', created_at), u.username, t.transaction_type
ORDER BY 1, total DESC;
```

→ ได้ผลลัพธ์ตรงกับรูป "Month view" ของลูกค้า

SECTION 06

Code ที่ต้องแก้ — แค่ 2 จุด

Chat handler + Admin edit user = 2 จุด INSERT

ทุกทีอื่นในระบบไม่ต้องแตะ — schema ที่มีอยู่ไม่เปลี่ยน, การคำนวณ cost / pricing / daily_usage ทำงานเหมือนเดิม.

☐ Chat handler — หลัง deduct credit

```
// (server.js · █ /api/chat ████ UPDATE tbl_credits)
await pool.query(`
  INSERT INTO tbl_user_credit_transaction
    (user_id, project_id, transaction_type, amount,
     balance_before, balance_after, ref_type, ref_id, created_by)
  VALUES ($1, $2, 'usage', $3, $4, $5, 'chat', $6, NULL)
`, [userId, projectId, -cost, balBefore, balAfter, chatSessionId]);
```

□ Admin edit user — เมื่อ balance เปลี่ยน

```
// (server.js · ■■ PUT /api/users/:id ■■■■■■■■ balance ■■ body)
const delta = balanceNum - prevBalance;
if (delta !== 0) {
  const txType = delta > 0 ? 'topup' : 'adjustment';
  await pool.query(`
    INSERT INTO tbl_user_credit_transaction
      (user_id, project_id, transaction_type, amount,
       balance_before, balance_after, ref_type, created_by)
    VALUES ($1, $2, $3, $4, $5, $6, 'admin_edit', $7)
  `, [userId, projectId, txType, delta, prevBalance, balanceNum,
    req.session.userId]);
}
```

รายการที่ต้อง approve

ก่อน implement ขอ confirm 8 ข้อนี้ — ถ้า OK ทั้งหมด ผมเริ่มสร้าง migration ได้เลย. ถ้าอยากปรับข้อใด ระบุเลข item + รายละเอียดใหม่ที่ต้องการ.

#	ประเด็น	ที่ตั้งใจ	OK?
1	ชื่อ table	tbl_user_credit_transaction	☐
2	Transaction types	4 ค่า — topup, usage (active) + adjustment, refund (reserve)	☐
3	Amount sign	Signed — + = inflow, - = outflow	☐
4	Backfill historic	YES — usage จาก tbl_response, topup จาก audit log	☐
5	balance_before/after (historic)	ปล่อย NULL ได้ (reconstruct ไม่ได้)	☐
6	Code changes	2 จุด — chat handler + admin edit user	☐
7	VIEW v_user_credit_transaction	สร้างเลย (JOIN user + project, friendly columns)	☐
8	UI Dashboard	ยังไม่ทำในรอบนี้ — แค่ schema + backend	☐

ตอบกลับ "ลุย" หรือ "approve" เพื่อเริ่ม implement.

ถ้ามีข้อที่ต้องการเปลี่ยน ระบุ เลข item + คำใหม่ ผมจะ update proposal ก่อน apply.